Aggregation and Composition in Java:

In Java, **association** refers to the relationship between two objects. Aggregation and Composition are specific types of association that represent different levels of dependency between objects.

---

1. Aggregation

Aggregation is a type of association that represents a **"whole-part"** relationship but with **loose coupling**. In aggregation, the child (part) can exist independently of the parent (whole) object. This means the life cycle of the part object is not dependent on the life cycle of the whole object.

Characteristics of Aggregation:

- **Loose coupling**: The child object can exist without the parent.
- **Independent life cycle**: The part object can exist without the whole object, and vice versa.
- **Reusability**: The child objects can be shared between multiple parent objects.

Example of Aggregation:

A university has many students, but a student can exist independently of the university.

Code Example of Aggregation:

```
class Student {
    private String name;

    public Student(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

class University {
    private String name;
    private List<Student> students;

    public University(String name) {
        this.name = name;
        students = new ArrayList<>();
    }

    public void addStudent(Student student) {
        students.add(student);
```

```
    }

    public void showStudents() {
        for (Student student : students) {
            System.out.println(student.getName());
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Alice");
        Student s2 = new Student("Bob");

        University university = new University("XYZ University");
        university.addStudent(s1);
        university.addStudent(s2);

        university.showStudents();
    }
}
```

In this example:

- The `University` class aggregates multiple `Student` objects.
- `Student` objects can exist independently of the `University`, so if a university object is deleted, the student objects can still exist.

---

2. Composition

Composition is a type of association that represents a **stronger "whole-part"** relationship. In composition, the child (part) object is dependent on the parent (whole) object. If the parent object is destroyed, the child objects are also destroyed. Composition implies a **stronger coupling** than aggregation.

Characteristics of Composition:

- **Strong coupling**: The part object cannot exist without the parent object.
- **Dependent life cycle**: The life cycle of the part object is tied to the life cycle of the whole object.
- **Exclusive ownership**: The part object is owned exclusively by the parent object, meaning it cannot be shared with other objects.

Example of Composition:

A house contains rooms, but rooms cannot exist without the house. If the house is destroyed, the rooms are also destroyed.

Code Example of Composition:

```
class Room {
    private String type;

    public Room(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }
}

class House {
    private String address;
    private Room room;

    public House(String address, String roomType) {
        this.address = address;
        this.room = new Room(roomType);  // Room is created inside House,
cannot exist without it
    }

    public void showRoomDetails() {
        System.out.println("House Address: " + address);
        System.out.println("Room Type: " + room.getType());
    }
}

public class Main {
    public static void main(String[] args) {
        House house = new House("123 Elm Street", "Bedroom");
        house.showRoomDetails();
    }
}
```

In this example:

- The `House` class has a composition relationship with the `Room` class.
- The `Room` object is created inside the `House` object, and cannot exist without it. If the `House` is deleted, the `Room` object will also be deleted because of the dependent life cycle.

Comparison of Aggregation and Composition

Aspect	Aggregation	Composition
Dependency	Loose coupling (part can exist independently)	Strong coupling (part cannot exist without whole)
Life Cycle	Independent life cycle for part and whole	Dependent life cycle (part is destroyed with whole)
Example	University and Students	House and Room
Ownership	Part can be shared by other objects	Part is owned exclusively by the whole object

---

Visual Representation of the Association

- **Aggregation**: The part object can exist independently of the whole object.
- **Composition**: The part object depends entirely on the whole object.

For example, in **aggregation**, a university may have students (each student may belong to multiple universities), but in **composition**, a house has rooms, and if the house is destroyed, the rooms are destroyed as well.
